TP — Communication REST entre micro-services

Partie 1

Objectifs

- Dans ce TP, on souhaite faire communiquer deux micro-services à l'aide d'appels HTTP/REST. On utilisera deux micro-services :
  - `produit-service` : fournit des informations sur les produits ;
  - `client-service` : interroge `produit-service`.
- L'objectif est de comprendre le principe suivant :

```
Client
   |
   v
client-service
   |
   | HTTP / REST
   v
produit-service
```

0.1 Le micro-service produit

- Créer un nouveau projet Spring Boot nommé `produit-service`. Le micro-service écoute sur le port 8080
- Créer un contrôleur REST proposant l'URL :

  `/produits/{id}`

- Pour simplifier, utiliser quelques produits définis directement dans le programme. Par exemple :

  `/produits/1`
  `/produits/2`
  `/produits/3`

- Une requête sur :

  `http://localhost:8080/produits/1`

  pourra retourner :

```
{
    "id": 1,
    "nom": "Ordinateur",
    "prix": 1200
}
```

- Répondre aux questions suivantes :

  1. Créer le contrôleur REST.
  2. Créer une méthode permettant de rechercher un produit à partir de son identifiant.
  3. Tester le service avec un navigateur ou `curl`. Par exemple :

     ```
     curl http://localhost:8080/produits/1
     ```

0.2 Le micro-service client

- Créer un deuxième projet Spring Boot nommé `client-service`. Ce micro-service doit utiliser le port 8081
- Créer un contrôleur proposant :

  ```
  /client/produit/{id}
  ```

  Par exemple :

  ```
  http://localhost:8081/client/produit/1
  ```

0.3 Faire communiquer les deux micro-services

- Le contrôleur de `client-service` doit effectuer une requête HTTP vers :

  ```
  http://localhost:8080/produits/{id}
  ```

- Il récupère la réponse de `produit-service` et la retourne au client.

  Le schéma de communication devient :

```
GET /client/produit/1
         |
         v
+----------------+
| client-service |
|     :8081      |
+----------------+
         |
         | GET /produits/1
         v
+----------------+
| produit-service|
|     :8080      |
+----------------+
         |
         v
{ id, nom, prix }
```

0.4 Tester la communication

- Lancer les deux micro-services.

  ```
  produit-service   : 8080
  client-service    : 8081
  ```

- Tester :

```
curl http://localhost:8081/client/produit/1
```

- Répondre aux questions suivantes :

  1. Quel micro-service reçoit directement la requête du client ?
  2. Quel micro-service fournit réellement les informations sur le produit ?
  3. Combien de requêtes HTTP sont effectuées ?
  4. Qui est le client HTTP et qui est le serveur HTTP ?
  5. Que se passe-t-il si `produit-service` est arrêté ?

0.5 Ajouter une création de produit

- Modifier `produit-service` afin d'ajouter une opération REST permettant de créer un produit. Utiliser une requête :

  ```
  POST /produits
  ```

  avec, par exemple :

  ```
  {
      "id": 4,
      "nom": "Telephone",
      "prix": 800
  }
  ```

- Répondre aux questions suivantes :

  1. Quelle annotation Spring permet de traiter une requête `POST` ?
  2. Quelle annotation permet de récupérer le contenu JSON envoyé dans la requête ?
  3. Quelle différence faites-vous entre `GET` et `POST` ?

Partie 2

- Dans cette partie, on s'intéresse au problème classique des 5 philosophes. On veut définir 5 micro-services, un par philosophe, et un 6ème, leur superviseur, qui contrôle les fourchettes. Sur la console de chacun, le philosophe affiche son état (réfléchir, attendre, manger) et se retire (fin) du micro-service. Quand tous les philosophes ont terminé de manger, le superviseur affiche l'ordre dans lequel les philosophes ont été servis. La communication d'un philosophe avec le serveur se fait via une API REST.
- Reprendre le problème des philosophes en proposant une version distribuée, c-a-d vous supprimez le superviseur. Chaque philosophe doit communiquer directement avec ses voisins.
- Discuter le phénomène d'interblocage et de famine.